

接

算力中心与工具平台技术文档

算力中心 · 工具平台 (AI 智能体生态)

「接单吧」OPC 撮合交易平台 · 技术介绍文档

版本: v1.0 | 编写日期: 2026 年 8 月 14 日 | 面向读者: 技术与产品团队、对外合作伙伴

目录

概述

第一章 算力中心

- 1.1 功能定位与业务价值
- 1.2 功能子模块与生命周期状态机
- 1.3 计费机制（按秒计费、账单水位、余额模型）
- 1.4 并发一致性设计（调度器、事务与锁）
- 1.5 前后端交互方式（接口清单）
- 1.6 数据存储结构

第二章 工具平台

- 2.1 功能定位与业务价值
- 2.2 子模块总览（工作区 / 市场 / 收益账单）
- 2.3 智能体使用与对话机制（权限判定、角色约束、会话存储）
- 2.4 付费订阅闭环（下单 → 支付核验 → 激活 → 收益记账）
- 2.5 前后端交互方式（接口清单）
- 2.6 数据存储结构
- 2.7 权限控制与并发保护

概述

「接单吧」在核心撮合交易能力之外，为平台用户（OPC 与发单方）构建了两大自助式生产力模块：

模块	解决的问题	技术形态
算力中心	AI 从业者需要随开随用的开发环境、训练与推理算力，且费用要按实际用量透明结算	Notebook / 训练任务 / 推理服务三类可计费资源的全生命周期管理，后台调度器按秒计费，账单流水与资源水位同事务原子记账
工具平台	创作者希望把自己的 AI 能力（智能体）沉淀为可复用、可变现的产品；使用者希望即订即用	智能体创建 → 发布市场 → 订阅付费 → 在线对话使用的完整闭环，LLM 调用统一封装并支持多供应商故障转移

两个模块运行在与主站一致的技术栈上：后端 Express + Drizzle ORM + PostgreSQL（`artifacts/api-server`），前端 React + Vite（`artifacts/jiedanba`）；金额一律以**人民币分**（`amountFen`）整数存储；时间遵循「北京墙上时间裸存储」约定。大模型调用统一封装在 `src/lib/llm.ts`，支持后台配置多个 OpenAI 兼容供应商（如 DeepSeek），按激活状态排序自动故障转移（failover）。

术语速查：**Notebook**=在线交互式开发环境（类 Jupyter）；**推理服务**=把训练好的模型部署为在线 API；**副本（replica）**=同一推理服务并行运行的实例数；**水位（lastBilledAt）**=资源「已计费到哪个时间点」的游标；**咨询锁（advisory lock）**=PostgreSQL 提供的应用级互斥锁；**FOR UPDATE**=行级排他锁，事务内锁定所选行防止并发修改；**LLM**=大语言模型。

第一章 算力中心

1.1 功能定位与业务价值

算力中心是面向 AI 从业者的自助算力控制台：用户可以一键创建在线开发环境（Notebook）、提交模型训练任务、部署推理服务，并配套管理存储、镜像、API 密钥、代币资源与个人模型/数据集仓库。所有可计费资源**按秒累计、按实际运行时长扣费**，账单流水实时可查。

业务价值：① 把「买卡、装环境、算账」的重资产流程变成开箱即用的云上自助服务；② 按量计费 + 透明账单降低使用门槛与信任成本；③ 与工具平台共享账号体系，构成「算力 → 模型 → 智能体产品化」的完整链路。

1.2 功能子模块与生命周期状态机

子模块	生命周期	说明
Notebook 开发环境	<code>creating</code> → （调度器约 30 秒后） <code>running</code> → <code>stopped</code> （可再 start）	可选环境类型 / 镜像 / 资源规格 / SSH 直连；重复 start 返回 409；停止与删除运行中的实例都会 先结算再变更
训练任务	<code>pending</code> → <code>running</code> → <code>completed</code> （到达计划时长自动完成并结清） / <code>stopped</code> （手动停止）	支持自定义命令、数据/输出路径；支持一键克隆（克隆为新的 pending 任务）；同步累计 <code>totalRuntimeSeconds</code>
推理服务	<code>deploying</code> → <code>running</code> （副本全部就绪） ⇄ <code>stopped</code>	指定模型来源、规格与副本数；运行时费用按副本数放大；停止后副本归零
存储	常驻 <code>running</code>	文件/对象存储的容量与用量管理（元数据）
计算资源实例	常驻 <code>running</code> ，支持到期时间	包周期 GPU/CPU 实例的登记管理
代币资源	额度用尽或到期	大模型 Token 包：总量 / 已用量 / 有效期
API 密钥	创建 / 删除	生成 <code>sk-</code> 前缀密钥；数据库 只存 SHA-256 哈希与展示前缀 ，明文仅在创建响应中一次性返回
镜像 / 仓库 / 收藏	—	自定义镜像登记；个人模型/数据集/镜像仓库（公开或私有）；收藏支持 toggle，私有且非本人的目标不暴露内容

状态推进由谁完成？ 创建后的 `creating/pending/deploying` → `running` 与训练任务的自动完成，均由后台**调度器**（每 30 秒一个 tick）推进，模拟真实云平台的异步就绪过程；用户侧的 start/stop/delete 则通过 REST 接口即时生效。

1.3 计费机制

1.3.1 计费单位与价格来源

所有金额以人民币分存储。运行费用按「秒数 × 小时单价 ÷ 3600」向上取整到分：`ceil(seconds × rateFenPerHour / 3600)`。小时单价由资源规格字符串匹配得出：

规格关键字	小时单价	说明
H100 / H800	24.00 元/小时	旗舰训练卡
A100 / A800	16.00 元/小时	主流训练卡
V100 / 4090 / 3090 / L20 / L40 及其他 GPU	8.00 元/小时	通用 GPU
纯 CPU 规格	1.00 元/小时	—

推理服务的费用再乘以副本数（取运行中副本数，至少按 1 计）。

1.3.2 账单与水位：原子记账模型

1. 每个计费资源带一个**水位字段** `lastBilledAt`，表示「已经结算到哪个时刻」。
 2. 调度器每个 tick 对每满 60 秒未结算的运行中资源执行 `billSegment`：在**同一个数据库事务**内插入一条 `compute_bills` 支出流水，并把水位推进到本次结算点。
 3. 用户停止 / 删除运行中的资源时，路由先对该行加 `FOR UPDATE` 行锁，把「水位 → 当前时刻」这一段**先结清**，再改状态或删除——保证既不多扣（重复结算）也不漏扣（未结算就删）。
 4. 训练任务完成 / 资源停止后水位置空，重新启动时以新的启动时间为新计费起点。

1.3.3 余额与订单

账户余额不是独立字段，而是账单流水的实时聚合：`余额 = Σ收入(income) - Σ支出(expense)`，由 `GET /compute/account` 返回。充值以 `itemType=recharge` 的收入流水入账。`compute_orders` 记录资源开通订单（CO 前缀单号），与账单按用户 / 类型 / 金额 / 时间勾稽。

已知演进方向：当前运行账单不直接关联具体资源实例与计费区间，按实例逐段追溯需靠时间与类型推断；账单-实例关联字段已列入产品迭代计划。

1.4 并发一致性设计

场景	机制
多实例部署下调度器重复执行	tick 事务内先抢 <code>pg_try_advisory_xact_lock</code> （事务级咨询锁），抢不到直接跳过——同一时刻全集群只有一个调度器在记账；事务结束锁自动释放，无泄漏风险
调度器与用户操作争抢同一资源	调度器选取候选行用 <code>FOR UPDATE SKIP LOCKED</code> （跳过被路由锁住的行）；路由 stop/delete 用 <code>FOR UPDATE</code> 锁行后先结算再变更
重复 start 重置计费起点	start 使用「状态前置条件更新」（仅当前状态允许才更新），重复 start 返回 409，不会覆盖水位
账单与水位不一致	两者永远在同一事务内提交，全有或全无

1.5 前后端交互方式

全部接口挂载于 `/api`，均要求登录（`requireAuth`）并按当前用户隔离数据；前端在 `components/compute/api.ts` 中统一封装带 Bearer Token 的请求方法。

接口	说明
<code>GET/POST /compute/notebooks</code> , <code>PATCH/DELETE /:id</code> , <code>POST /:id/start stop</code>	Notebook 全生命周期
<code>GET/POST /compute/training-jobs</code> , <code>PATCH/DELETE /:id</code> , <code>POST /:id/stop clone</code>	训练任务；clone 复制配置为新 pending 任务
<code>GET/POST /compute/inference-services</code> , <code>PATCH/DELETE /:id</code> , <code>POST /:id/start stop</code>	推理服务
<code>GET/POST/PATCH/DELETE /compute/{storages resources images}</code>	存储 / 计算资源 / 镜像管理
<code>GET/POST /compute/token-resources</code>	代币资源
<code>GET/POST /compute/api-keys</code> , <code>DELETE /:id</code>	API 密钥；创建时一次性返回明文
<code>GET/POST /compute/orders</code> 、 <code>GET /compute/bills</code> 、 <code>GET /compute/account</code>	订单、账单流水、余额
<code>GET/POST /compute/repo-items</code> , <code>DELETE /:id</code> ; <code>GET /compute/favorites</code> , <code>POST /compute/favorites/toggle</code>	个人仓库与收藏
<code>GET /compute/overview</code>	总览：四类资源统计 + 最近订单与账单

1.6 数据存储结构

表	关键字段	说明
<code>compute_notebooks</code>	status、envType、image、resourceSpec、sshEnabled、startedAt/stoppedAt、 lastBilledAt 、totalRuntimeSeconds	开发环境；水位字段驱动按秒计费
<code>compute_training_jobs</code>	status、mode、command、datasetPath/outputPath、plannedDurationSeconds、lastBilledAt、totalRuntimeSeconds	训练任务；计划时长到达自动完成
<code>compute_inference_services</code>	status、modelSource、resourceSpec、replicas/runningReplicas、endpointUrl、lastBilledAt	推理服务；费用按副本数放大
<code>compute_storages</code> / <code>compute_resources</code> / <code>compute_token_resources</code>	容量与用量 / GPU 规格与到期时间 / token 总量与已用量	配套资源
<code>compute_api_keys</code>	keyPrefix、 keyHash (SHA-256)、lastUsedAt	不存明文密钥
<code>compute_images</code> / <code>compute_repo_items</code> / <code>compute_favorites</code>	镜像 source/tag/size；仓库 repoType(model/dataset/image)、visibility、tags、downloads	镜像与个人市场
<code>compute_orders</code>	orderNo (CO 前缀)、itemType、amountFen、status	开通订单
<code>compute_bills</code>	billNo、itemType、amountFen、 direction(income/expense)	资金流水；余额=收入-支出的聚合

第二章 工具平台

2.1 功能定位与业务价值

工具平台是平台内的 **AI 智能体生态**：创作者把自己的专业能力封装成「只做一件事」的智能体（如合同风险审查、会议纪要整理、AI 生图方案、编程助手），发布到市场按月订阅变现；使用者订阅后在线对话使用，会话历史长期留存。

业务价值：① 为 OPC / 创作者提供第二收入曲线（订阅分成）；② 使用者以极低成本获得垂直场景的 AI 生产力；③ 订阅支付、收益记账与主站支付体系复用同一套网关与核验逻辑，账实一致。

2.2 子模块总览

分区	子模块	能力
工作区	我的智能体	聚合「已订阅」+「我创建的」两类可用智能体，一键进入使用页
	智能体管理	创建 / 编辑 / 发布（免费或付费） / 下架 / 发布为模板
	知识库	知识库元数据管理（名称、标签、容量、文档数）
	工具管理	自定义工具的配置管理（kind + JSONB config + 启用开关）
市场	智能体市场	浏览已发布智能体；详情页含作者、订阅数、收藏数；收藏 / 订阅 / 立即使用
	模板中心	模板一键添加到自己的工作区
	插件市场	插件安装与安装量统计
收益与账单	创作者收益	本人智能体的订阅收益流水
	订阅与账单	本人订阅列表、历史支付流水、累计支出；续费 / 退订 / 使用入口

2.3 智能体使用与对话机制

2.3.1 使用权限判定（canUseAgent）

每次对话请求都在服务端重新判定使用权，**不信任前端状态**。满足其一即可使用：① 本人创建的智能体；② 已发布且免费；③ 存在 `active` 且未过期的订阅。付费未订阅用户收到 403，前端引导「前往市场订阅」。

2.3.2 角色约束（buildSystemPrompt）

系统提示词把智能体约束为「只做一件事的专业工具」而非通用聊天助手：信息不足时列出所需材料与输入示例；产出结构化交付物（表格 / 分级 / 编号清单）；workflow 类型分步展示执行过程；与职责无关的请求一律拒绝并引导回本职；全程简体中文。这保证了不同智能体有明确差异化的产品体验。

2.3.3 对话流程与会话存储

1. 前端 `POST /tools/agents/:agentId/chat` (可携带 `conversationId` 续聊)。
2. 服务端校验使用权后组装上下文：系统提示词 + 最近 20 条历史消息 (单条上限 4000 字)，调用 `callLLM` (多供应商故障转移)。
3. **LLM 调用期间不持有任何数据库锁**；拿到回复后开启短事务，对会话行 `FOR UPDATE` **重读最新消息数组再追加**本轮问答——并发续聊场景下消息永不丢失 (会话已被删则返回 404)。
4. 新会话自动以首条消息前 30 字为标题；全部消息 (含时间戳) 以 JSONB 数组持久化，跨设备、跨会话可回看、可续聊、可删除。
5. 前端使用页：左侧历史会话栏 (点开续聊 / 删除)，右侧聊天区乐观展示用户消息；发送失败自动回退并找回未发送文本；消息内容支持图片语法渲染 (生图类智能体的产出直接以图片展示)。

2.4 付费订阅闭环

下单 → 支付 → 核验 → 激活 → 记账 全链路

1. **下单**：短事务锁定该用户-智能体的唯一订阅行，写入 `pending_payment` 状态、冻结应付金额与业务单号 (`TOOLSUB-` 前缀)；意向单先落库，避免「网关成功但本地无主订单」。
2. **调起支付**：事务外调用支付网关创建支付单，再以短事务回填 `paymentOrderNo`；未支付的网关订单可复用，已终结的订单先清链再重建。
3. **核验**：激活前服务端主动调 `queryPaymentStatus` 查单，必须为 PAID 且**网关金额与冻结金额严格相等**——不信任回调报文，也不信任前端。
4. **激活**：事务内对订阅行 `FOR UPDATE`，仅 `pending_payment` 且订单号匹配才转 `active`，生效期为支付时刻起 30 天；支付回调与前端轮询可能并发到达，行锁 + 状态门槛保证**只激活一次**。
5. **记账**：同一事务写入一条不可变的支付流水 (`tool_subscription_payments`，唯一冲突忽略保证幂等) 与一条创作者收益 (`tool_earnings`)。免费订阅直接激活、长期有效。

账务口径：订阅行是「当前状态」(每用户每智能体一行，续订复用更新)；**累计支出以支付流水表为事实源**，历史每笔成功支付均独立留痕，不随续订覆盖。到期在查询时惰性标记 `expired`；待支付订单不可退订。

2.5 前后端交互方式

全部接口要求登录；市场详情对非公开且非本人的智能体返回 404 (不暴露资源存在性)。

接口	说明
GET/POST /tools/agents , PATCH/DELETE /:id , POST /:id/publish unpublish publish-template	智能体管理与上下架（仅 owner）
GET /tools/market 、 GET /tools/market/:agentId	市场列表与详情（作者、订阅数、收藏数、是否可用）
POST /tools/market/:agentId/favorite subscribe payment-status unsubscribe	收藏 toggle、订阅下单、支付核验轮询、退订
POST /tools/agents/:agentId/chat	对话（新聊 / 续聊），服务端二次权限校验
GET /tools/agents/:agentId/conversations , GET/DELETE /tools/agent-conversations/:id	使用历史列表 / 会话详情 / 删除（仅本人）
GET/POST/PATCH/DELETE /tools/knowledge-bases 、 /tools/custom-tools	知识库与自定义工具（本人 CRUD）
GET /tools/templates , POST /:id/add-to-workspace ; GET /tools/plugins , POST /:id/install	模板复制到工作区；插件安装
GET /tools/earnings 、 GET /tools/subscriptions	创作者收益；本人订阅 + 历史支付 + 累计支出

2.6 数据存储结构

表	关键字段	说明
<code>tool_agents</code>	ownerId、name、appType(agent/workflow)、description、tags[]、category、 shareStatus (private/published/template)、 priceFenPerMonth 、publishedAt	智能体主表；价格为 0 即免费
<code>tool_agent_conversations</code>	userId、agentId、title、 messages(JSONB 数组：role/content/at)	使用会话；追加走行锁重读
<code>tool_subscriptions</code>	userId+agentId 唯一、 status(pending_payment/active/cancelled/expired)、 amountFen、businessOrderNo、paymentOrderNo、 paidAt、startsAt/expiresAt	订阅当前状态（一人一智能体一行）
<code>tool_subscription_payments</code>	subscriptionId、amountFen、业务/支付单号、paidAt	不可变成功支付历史，幂等写入
<code>tool_earnings</code>	ownerId、agentId、subscriberId、amountFen	创作者收益流水
<code>tool_agent_favorites</code> / <code>tool_plugins</code> / <code>tool_plugin_installs</code>	收藏关系；插件与安装记录、installCount	市场互动
<code>tool_knowledge_bases</code> / <code>tool_custom_tools</code>	知识库元数据；工具 kind + config(JSONB) + enabled	工作区配套资产

2.7 权限控制与并发保护

控制点	规则
数据隔离	所有列表 / 详情 / 会话按当前用户或 owner 过滤；非公开智能体对外 404
使用权二次校验	对话接口独立执行 <code>canUseAgent</code> ，不依赖前端传入的可用状态
支付防重	订阅行锁 + 状态门槛 + 支付流水唯一冲突忽略，回调与轮询并发只激活一次；金额服务端严格核对
对话并发	LLM 调用不持锁；落库阶段 <code>FOR UPDATE</code> 重读追加，消息零丢失
输入约束	单条消息 4000 字上限；上下文取最近 20 条；非法 ID（非纯数字）返回 404/400 而非 500
已知演进方向	收益行暂未冗余支付单外键（逐笔对账需经订阅+流水中转）；插件暂无卸载入口——均已列入产品迭代计划